2080.5
Рейтинг

RUVDS.com

VDS/VPS-хостинг. Скидка 15% по коду **HABR15**

ru_vds 17 авг 2018 в 13:08

Var, let или const? Проблемы областей видимости переменных и ES6

13 мин 94K

Блог компании RUVDS.com, Разработка веб-сайтов*, JavaScript*

Перевод

Автор оригинала: Brandon Gregory

Области видимости в JavaScript всегда были непростой темой, особенно в сравнении с более строго организованными языками, такими, как C и Java. В течение многих лет области видимости в JS особенно широко не обсуждались, так как в языке попросту не было средств, которые позволяли бы существенно повлиять на сложившуюся ситуацию. Но в ECMAScript 6 появились некоторые новые возможности, которые позволяют разработчикам лучше контролировать области видимости переменных. Эти возможности в наши дни уже очень хорошо поддерживают браузеры, они вполне доступны для большинства разработчиков. Однако новые ключевые слова

для объявления переменных, учитывая ещё и то, что старое ключевое слово `var` никуда не делось, означают не только новые возможности, но и появление новых вопросов. Когда использовать ключевые слова `let` и `const`? Как они себя ведут? В каких ситуациях всё ещё актуально ключевое слово `var`? Материал, перевод которого мы сегодня публикуем, направлен на исследование проблемы областей видимости переменных в JavaScript.

Области видимости переменных: обзор

Область видимости переменной — это важная концепция в программировании, которая, правда, может запутать некоторых разработчиков, особенно — новичков. Область видимости переменной

Взгляните на следующий пример:

```
var myVar = 1;

function setMyVar() {
  myVar = 2;
}

setMyVar();

console.log(myVar);
```

Что выведет метод `console.log`? Ответ на этот вопрос никого не удивит: он выведет `2`. Переменная `myVar` объявлена за пределами какой-либо функции, что говорит нам о том, что она объявлена в глобальной области видимости. Следовательно, любая функция, объявленная в той же области видимости, сможет обратиться к `myVar`. На самом деле, если речь идёт о коде, выполняемом в браузере, доступ к этой переменной будет даже у функций, объявленных в других файлах, подключённых к странице.

Теперь взглянем на следующий код:

```
function setMyVar() {
  var myVar = 2;
}

setMyVar();
```

```
console.log(myVar);
```

Внешне его изменения, по сравнению с предыдущим примером, незначительны. А именно, мы всего лишь поместили объявление переменной внутрь функции. Что теперь выведет `console.log`? На самом деле, ничего, так как эта переменная не объявлена и при попытке обратиться к ней будет выведено сообщение о необработанной ошибке `ReferenceError`. Произошло так из-за того, что переменную, с помощью ключевого слова `var`, объявили внутри функции. В результате область видимости этой переменной ограничивается внутренней областью видимости функции. К ней можно обратиться в теле этой функции, с ней могут работать функции, вложенные в эту функцию, но извне она недоступна. Если нам надо, чтобы некоей переменной могли бы пользоваться несколько функций, находящихся на одном уровне, нам надо объявлять эту переменную там же, где объявлены эти функции, то есть — на один уровень выше их внутренних областей видимости.

Вот одно интересное наблюдение: код большинства веб-сайтов и веб-приложений не относится к творчеству какого-то одного программиста. Большинство программных проектов являются результатами командной разработки, и, кроме того, в них используются сторонние библиотеки и фреймворки. Даже если разработкой некоего сайта занимается один программист, обычно он пользуется внешними ресурсами. Из-за этого обычно не рекомендуется объявлять переменные в глобальной области видимости, так как нельзя заранее знать, какие переменные будут объявлять другие разработчики, код которых будет использоваться в проекте. Для того чтобы обойти эту проблему, можно использовать некоторые приёмы, в частности — паттерн «Модуль» и IIFE при применении объектно-ориентированного подхода к JavaScript-разработке, хотя того же эффекта позволяет достичь инкапсуляция данных и функций в обычных объектах. В целом же можно отметить, что переменные, область видимости которых выходит за пределы той, которая им необходима, обычно представляют собой проблему, с которой надо что-то делать.

Проблема ключевого слова var

Итак, мы разобрались с понятием «область видимости». Теперь перейдём к более сложным вещам. Взгляните на следующий код:

```
function varTest() {
  for (var i = 0; i < 3; i++) {
    console.log(i);
  }
  console.log(i);
}

varTest();
```

Что попадёт в консоль после его выполнения? Понятно, что внутри цикла будут выводиться значения увеличивающегося счётчика `i` : `0` , `1` и `2` . После того, как цикл завершается, программа продолжает выполняться. Теперь мы пытаемся обратиться к той же самой переменной-счётчику, которая была объявлена в цикле `for` , за пределами этого цикла. Что из этого выйдет?

В консоль, после обращения к `i` за пределами цикла, попадёт 3, так как ключевое слово `var` действует на уровне функции. Если объявить переменную с использованием `var` , то обратиться к ней в функции можно и после выхода из той конструкции, где она была объявлена.

Это может превратиться в проблему тогда, когда функции усложняются. Рассмотрим следующий пример:

```
function doSomething() {
  var myVar = 1;
  if (true) {
    var myVar = 2;
    console.log(myVar);
  }
  console.log(myVar);
}

doSomething();
```

Что попадёт в консоль теперь? `2` и `2`. Мы объявляем переменную, инициализируем её числом 1, а затем пытаемся переопределить ту же переменную внутри выражения `if`. Так как два эти объявления существуют в одной и той же области видимости, мы не можем объявить новую переменную с тем же именем, даже хотя мы, очевидно, хотим сделать именно это. В результате первая переменная перезаписывается внутри выражения `if`.

Вот в этом-то и заключается самый большой недостаток ключевого слова `var`. Область видимости переменных, объявленных с его использованием, оказывается слишком большой. Это может привести к непреднамеренной перезаписи данных и к другим ошибкам. Большие области видимости часто ведут к появлению неаккуратных программ. В целом, переменная должна иметь область видимости, ограниченную её нуждами, но не превышающую их. Хорошо было бы иметь возможность объявлять переменные, область видимости которых не так велика, как при использовании `var`, что позволило бы, при необходимости, применять более стабильные и лучше защищённые от ошибок программные конструкции. Собственно говоря, такие возможности нам даёт ECMAScript 6.

Новые способы объявления переменных

Стандарт ECMAScript 6 (новый набор возможностей JavaScript, известный ещё как ES6 и ES2015) даёт нам два новых способа объявления переменных, отличающихся ограниченной, по сравнению с `var`, областью видимости и имеющих ещё некоторые особенности. Это — ключевые слова `let` и `const`. И то и другое даёт нам так называемую блочную область видимости. Это означает, что область видимости при их использовании может быть ограничена блоком кода, таким, как цикл `for` или выражение `if`. Это даёт разработчику больше гибкости в выборе областей видимости переменных. Рассмотрим новые ключевые слова.

Использование ключевого слова let

Ключевое слово `let` очень похоже на `var`, основное отличие — ограниченная область видимости переменных, объявляемых с его помощью. Перепишем один из вышеприведённых примеров, заменив `var` на `let`:

```
function doSomething() {
  let myVar = 1;
  if (true) {
    let myVar = 2;
    console.log(myVar);
  }
  console.log(myVar);
}

doSomething();
```

В данном случае в консоль попадут числа `2` и `1`. Происходит это из-за того, что выражение `if` задаёт новую область видимости для переменной, объявленной с помощью ключевого слова

`let`. Это приводит к тому, что вторая объявленная переменная — это совершенно самостоятельная сущность, не связанная с первой. С ними можно работать независимо друг от друга. Однако это не значит, что вложенные блоки кода, вроде нашего выражения `if`, полностью отрезаны от переменных, объявленных с помощью ключевого слова `let` в той области видимости, в которой находятся они сами. Взгляните на следующий код:

```
function doSomething() {
  let myVar = 1;
  if (true) {
    console.log(myVar);
  }
}

doSomething();
```

В этом примере в консоль попадёт число `1`. У кода, находящегося внутри выражения `if` есть доступ к переменной, которую мы создали за его пределами. Поэтому он и выводит её значение в консоль. А что произойдёт, если попытаться перемешать области видимости? Например, сделать так:

```
function doSomething() {
  let myVar = 1;
  if (true) {
    console.log(myVar);
    let myVar = 2;
    console.log(myVar);
  }
}
```

```
doSomething();
```

Может показаться, что первый вызов `console.log` выведет `1`, но на самом деле при попытке выполнить этот код появится ошибка `ReferenceError`, которая сообщает нам о том, что переменная `myVar` для данной области видимости не определена или не инициализирована (текст этой ошибки различается в разных браузерах). В JavaScript существует такое явление, как поднятие переменных в верхнюю часть их области видимости. То есть, если в некоей области видимости объявляют переменную, JavaScript резервирует место для неё ещё до того, как будет выполнена команда её объявления. То, как именно это происходит, различается при использовании `var` и `let`.

Рассмотрим следующий пример:

```
console.log(varTest);
var varTest = 1;

console.log(letTest);
let letTest = 2;
```

В обоих случаях мы пытаемся воспользоваться в переменной до её объявления. Но команды вывода данных в консоль ведут себя по-разному. Первая, использующая переменную, которая позже будет объявлена с помощью ключевого слова `var`, выведет `undefined` — то есть то, что будет записано в эту переменную. Вторая же команда, которая пытается обратиться к переменной, которая позже будет объявлена с помощью ключевого слова `let`, выдаст `ReferenceError` и сообщит нам, что мы пытаемся использовать переменную до её объявления или инициализации. В чём дело?

А дело тут в том, что перед выполнением кода механизмы, ответственные за его выполнение, просматривают этот код, выясняют, будут ли в нём объявляться какие-то переменные, и, если это так, осуществляют их поднятие с резервированием места для них. При этом переменные, объявленные с помощью ключевого слова `var`, инициализируются значением `undefined` в пределах своей области видимости, даже если к ним обращаются до того, как они будут объявлены. Основная проблема тут заключается в том, что значение `undefined` в переменной не всегда указывает на то, что переменной пытаются воспользоваться до её объявления. Взгляните на следующий пример:

```
var var1;
console.log(var1);

console.log(var2);
var var2 = 1;
```

В данном случае, несмотря на то, что `var1` и `var2` объявлены по-разному, оба вызова `console.log` выведут `undefined`. Дело здесь в том, что в переменные, объявленные с помощью `var`, но не инициализированные, автоматически записывается значение `undefined`. При этом, как мы уже говорили, переменные, объявленные с помощью `var`, к которым обращаются до момента их объявления, так же содержат `undefined`. В результате, если в подобном коде что-то пойдёт не так, нельзя будет понять, что именно является источником ошибки — использование неинициализированной переменной или использование переменной до её объявления.

Место для переменных, объявляемых с помощью ключевого слова `let`, резервируется в их блоке, но, до их объявления, они попадают во временную мёртвую зону (TDZ, Temporal Dead

Zone). Это приводит к тому, что ими, до их объявления, пользоваться нельзя, а попытка обращения к такой переменной приводит к ошибке. Однако система точно знает причину проблемы и сообщает об этом. Это хорошо видно на данном примере:

```
let var1;
console.log(var1);

console.log(var2);
let var2 = 1;
```

Здесь первый вызов `console.log` выведет `undefined`, а второй вызовет ошибку `ReferenceError`, сообщая нам о том, что переменная пока не объявлена или не инициализирована.

В результате, если при использовании `var` появляется `undefined`, мы не знаем причину подобного поведения программы. Переменная может быть либо объявлена и неинициализирована, либо может быть ещё не объявлена в данной области видимости, но будет объявлена в коде, который расположен ниже команды обращения к ней. При использовании ключевого слова `let` мы можем понять — что именно происходит, а это гораздо полезнее для отладки.

Использование ключевого слова const

Ключевое слово `const` очень похоже на `let`, но у них есть одно важное различие. Это ключевое слово используется для объявления констант. Значения констант после их инициализации менять нельзя. Нужно отметить, что это относится лишь к значениям примитивных типов, воде строки или числа. Если константа является чем-то более сложным,

например — объектом или массивом, внутреннюю структуру подобной сущности модифицировать можно, нельзя лишь заменить её саму на другую. Взгляните на следующий код:

```
let mutableVar = 1;
const immutableVar = 2;

mutableVar = 3;
immutableVar = 4;
```

Этот код будет выполняться вплоть до последней строки. Попытка назначить новое значение константе приведёт к появлению ошибки `TypeError`. Именно так ведут себя константы, но, как уже было сказано, объекты, которыми инициализируют константы, можно менять, они могут подвергаться мутациям, что способно приводить к неожиданностям.

Возможно вам, как JavaScript-разработчику, интересно, почему иммутабельность переменных — это важно. Константы — это новое явление в JavaScript, в то время как они являются важнейшей частью таких языков, как C или Java. Почему эта концепция так популярна? Дело в том, что использование констант заставляет нас думать о том, как именно работает наш код. В некоторых ситуациях изменение значения переменной может нарушить работу кода, например, если в ней записано число Пи и к ней постоянно обращаются, или если в переменной имеется ссылка на некий HTML-элемент, с которым постоянно нужно работать. Скажем, вот константа, в которую записана ссылка на некую кнопку:

```
const myButton = document.querySelector('#my-button');
```

Если код зависит от ссылки на HTML-элемент, то нам нужно обеспечить неизменность этой

ссылки. В результате можно говорить, что ключевое слово `const` идёт не только по пути улучшений в сфере области видимости, но и по пути ограничения возможности модификации значений констант, объявленных с использованием этого ключевого слова. Вспомните, как мы говорили о том, что переменная должна иметь именно такую область видимости, которая ей нужна. Эту мысль можно продолжить, выдвинув рекомендацию, в соответствии с которой у переменной должна быть лишь такая возможность изменяться, которая нужна для правильной работы с ней, и не более того. Вот хороший материал на тему иммутабельности, из которого можно сделать важный вывод, в соответствии с которым использование иммутабельных переменных заставляет нас тщательнее обдумывать свой код, что ведёт к улучшению чистоты кода и к снижению числа неприятных неожиданностей, возникающих при его работе.

Когда я только начал использовать ключевые слова `let` и `const`, я, в основном, применял `let`, прибегая к `const` лишь тогда, когда запись нового значения в переменную, объявленную с помощью `let`, могла бы навредить программе. Но, всё больше узнавая о программировании, я изменил своё мнение по этому вопросу. Теперь мой основной инструмент — это `const`, а `let` я использую только тогда, когда значение переменной нужно перезаписывать. Это заставляет меня думать о том, действительно ли необходимо менять значение некоей переменной. В большинстве случаев делать этого не нужно.

Нужно ли нам ключевое слово var?

Ключевые слова `let` и `const` способствуют применению более ответственного подхода к программированию. Существуют ли ситуации, в которых всё ещё нужно ключевое слово `var`? Да, существуют. Есть несколько ситуаций, в которых это ключевое слово нам ещё пригодится. Как следует поразмыслите над тем, о чём мы сейчас поговорим, прежде чем менять `var` на `let` или `const`.

Уровень поддержки ключевого слова var браузерами

Переменные, объявленные с помощью ключевого слова `var`, отличаются одной очень важной особенностью, отсутствующей у `let` и `const`. А именно, речь идёт о том, что это ключевое слово поддерживают абсолютно все браузеры. Хотя поддержка браузерами let и const весьма хороша, однако, существует риск того, что ваша программа попадёт в браузер, их не поддерживающий. Для того чтобы понять последствия подобного происшествия, нужно учитывать то, как браузеры обходятся с неподдерживаемым JavaScript-кодом, в противовес, например, тому, как они реагируют на непонятный им CSS-код.

Если браузер не поддерживает какую-то возможность CSS, то это, в основном, ведёт к некоторым искажениям того, что будет выведено на экран. Сайт в браузере, который не поддерживает какие-то из используемых сайтом стилей, будет выглядеть не так, как ожидается, но им, весьма вероятно, можно будет пользоваться. Если же вы используете, например, `let`, а браузер это ключевое слово не поддерживает, то ваш JS-код там просто не будет работать. Не будет — и всё. Учитывая то, что JavaScript является одной из важных составных частей современного веба, это может стать серьёзнейшей проблемой в том случае, если вам надо, чтобы ваши программы работали бы в устаревших браузерах.

Когда говорят о поддержке сайтов браузерами, обычно задаются вопросом о том, в каком браузере сайт будет работать оптимально. Если же речь идёт о сайте, функционал которого основан на использовании `let` и `const`, то похожий вопрос придётся ставить иначе: «В каких браузерах наш сайт работать не будет?». И это — гораздо более серьёзно, чем разговор о том, использовать или нет `display: flex`. Для большинства веб-сайтов число пользователей с устаревшими браузерами не будет достаточно большим, чтобы об этом стоило бы беспокоиться. Однако если речь идёт о чём-то вроде интернет-магазина, или сайтов, владельцы которых покупают рекламу, это может быть весьма важным соображением. Прежде чем использовать новые возможности в подобных проектах, оцените уровень риска.

Если вам нужно поддерживать по-настоящему старые браузеры, но вы хотите при этом использовать `let`, `const` и другие новые возможности ES6, одним из вариантов решения проблемы является применение JavaScript-транспилятора наподобие Babel. Транспиляторы обеспечивают перевод нового кода в то, что будет понятно старым браузерам. Применяя Babel, можно писать современный код, использующий самые свежие возможности языка, а затем преобразовывать его в код, который могут выполнять устаревшие браузеры.

Звучит слишком хорошо, чтобы быть правдой? На самом деле, использование транспиляторов таит в себе некоторые неприятные особенности. Так, это значительно увеличивает объём готового кода, если сравнить его с тем, что можно было бы получить, написав его вручную. Как результат, растёт объём файлов. Кроме того, если вы начали использовать некий транспилятор, ваш проект оказывается привязанным к нему. Даже если вы пишете ES6-код, который совершенно правильно обрабатывается Babel, отказ от Babel приведёт к тому, что вам придётся перепроверять весь код, тщательно тестировать его. Если ваш проект работает как часы, эта идея вряд ли понравится тем, кто занимается его разработкой и поддержкой. Тут придётся задаваться некоторыми вопросами. Когда планируется переработать кодовую базу? Когда поддержка чего-то вроде IE8 уже не будет иметь значения? Возможно, ответы на эти вопросы и не повлияют на отказ от транспилятора, но их, в любом случае, надо учитывать, решаясь на столь серьёзный шаг.

Использование var для решения одной специфической задачи

Есть ещё одна ситуация, в которой ключевое слово `var` может то, чего не могут другие. Это довольно специфическая задача. Рассмотрим следующий код:

```
var myVar = 1;

function myFunction() {
```

```
  var myVar = 2;
  // Внезапно оказалось, что нам нужна переменная myVar из глобальной области видимости!
}
```

Итак, тут мы объявляем переменную `myVar` в глобальной области видимости, но позже теряем к ней доступ, так как объявляем такую же переменную в функции. Вдруг оказывается, что нам нужна и переменная из глобальной области видимости. Подобная ситуация может показаться надуманной, так как первую переменную, например, можно просто передать функции, или можно переименовать одну из них. Но вполне может случиться так, что ни то ни другое вам не доступно. И вот тут нам и пригодятся возможности `var`.

```
var myVar = 1;

function myFunction() {
  var myVar = 2;
  console.log(myVar); // 2
  console.log(window.myVar); // 1
}
```

Когда переменная объявляется в глобальной области видимости с использованием `var`, она автоматически привязывается к глобальному объекту `window`. Ключевые слова `let` и `const` этого не делают. Эта особенность однажды выручила меня в ситуации, когда сборочный скрипт проверял JS-код перед объединением файлов, а ссылка на глобальную переменную в одном из файлов (который, после сборки, был бы объединён с другими) выдавала ошибку, что не давало собрать проект.

Надо отметить, что использование этой возможности ведёт к написанию неаккуратного кода.

Чаще всего подобную проблему решают гораздо изящнее, получая в итоге более чистый код и меньшую вероятность возникновения ошибок. Речь идёт о том, что переменные, в виде свойств, записывают в собственный объект:

```
let myGlobalVars = {};
let myVar = 1;
myGlobalVars.myVar = myVar;

function myFunction() {
  let myVar = 2;
  console.log(myVar); // 2
  console.log(myGlobalVars.myVar); // 1
}
```

Безусловно, тут придётся писать больше кода, но этот подход делает код, используемый для решения неких неожиданных проблем, понятнее. В любом случае, бывает так, что рассматриваемая возможность ключевого слова `var` оказывается полезной, хотя, прежде чем прибегать к ней, стоит поискать другой, более понятный, способ решения проблемы.

Итоги

Итак, что выбрать? Как расставить приоритеты? Вот некоторые соображения по этому поводу:

- Вы собираетесь поддерживать IE10 или по-настоящему старые браузеры? Если вы даёте на этот вопрос положительный ответ и не собираетесь пользоваться транспиляторами — отказывайтесь от новых возможностей и используйте `var` .

- Если вы можете позволить себе использование новых возможностей JavaScript, начать стоит с того, чтобы везде, где раньше применялось ключевое слово `var`, использовать `const`. Если где-то нужна возможность перезаписи значения переменной (хотя, если вы попытаетесь переписать свой код, то эта возможность может вам и не понадобиться) — используйте `let`.

Новые ключевые слова `let` и `const`, появившиеся в ECMAScript 6, дают нам больше возможностей по контролю за областью видимости переменных (и констант) в коде веб-сайтов и веб-приложений. Они заставляют нас больше думать о том, как именно работает код, а такие размышления хорошо влияют на то, что у нас получается. Конечно, прежде чем воспользоваться чем-то новым, стоит взвесить все «за» и «против» в применении к конкретной задаче, но, используя `let` и `const`, вы сделаете свои проекты стабильнее и приготовите их к будущему.

Уважаемые читатели! Согласны ли вы с рекомендацией, в соответствии с которой ключевое слово `const` стоит сделать основной заменой ключевому слову `var`, прибегая к `let` лишь в тех случаях, когда значение переменной, по объективным причинам, нужно перезаписывать?

Habrahabr10 Промо-код для скидки в 10% на наши виртуальные сервера

Теги: JavaScript, разработка

Хабы: Блог компании RUVDS.com, Разработка веб-сайтов, JavaScript

Редакторский дайджест
Присылаем лучшие статьи раз в месяц